CHRISTOPHER NEWPORT UNIVERSITY · SCHOOL OF ENGINEERING & COMPUTING

Estimating LLM Inference

Memory Requirements

Memory & GPU planning for large-language-model inference

Keith Perkins · School of Engineering & Computing

WHY IT MATTERS

Why memory matters for LLM inference

In inference mode (not training), understanding memory consumption is critical for successful deployment. Under-estimating required memory can lead to OOM errors or inefficient resource allocation.

Model Weights

Parameters stored in GPU memory

Activation & Hidden States

Intermediate layer outputs maintained during forward pass computation

KV-Cache

Key-value tensors for autoregressive generation, growing with sequence length

Parallelism Overheads

Splitting a model across GPUs adds communication buffers and some duplicated layers

KEY FACTORS

Key factors influencing memory usage

Multiple dimensions affect the total memory footprint of your LLM deployment. Context length, model architecture, and parallelism strategies all compound to determine your GPU requirements.



Model Size

Parameter count directly sets weight memory: parameters × bytes per parameter



Context Length

KV-cache grows linearly with sequence length and dominates at long contexts



Batch Size

Each concurrent request needs its own KV-cache; weights are shared



GPU Parallelism

Splitting a model across GPUs adds communication buffers and some replicated layers

Memory breakdown components

Understanding each memory component helps you accurately estimate total GPU requirements and identify optimization opportunities.

1	**Model Weights**	Number of parameters × bytes per parameter (e.g., 2 bytes for FP16, 1 byte for INT8)
2	**Activation Buffers**	Intermediate outputs during the forward pass. They scale with batch size × sequence length, not with model size.
3	**KV-Cache Storage**	Key and value vectors for every layer and every token in context. **This is the big one.**
4	**Framework Overheads**	CUDA context, allocator fragmentation, communication buffers. Budget ~20% (with activations) on top of weights + KV-cache.

Model weights: precision matters

Let's walk through a concrete example to estimate weight memory for a 10 billion parameter model.

ASSUMPTIONS

- Model size: 10 billion parameters
- Precision: 16-bit floats (FP16)
- Storage: 2 bytes per parameter

CALCULATION

10 billion parameters × 2 bytes

Weights = 20 GB

OPTIMIZATION: QUANTIZATION

Storing weights in fewer bits shrinks them proportionally:

INT8: 10 billion × 1 byte = **10 GB**

4-bit (GPTQ, AWQ): 10 billion × 0.5 byte = **5 GB**

50% smaller weights at INT8, with minimal accuracy impact for many use cases. 4-bit saves more but costs more accuracy.

KV-cache memory

The KV-cache often dominates memory usage for long-context inference. Here's how to estimate it.

STEP 1

Architecture parameters

Context length: 2048 tokens

Layers: 40

Hidden size: 4096 = 32 heads × 128

KV heads: 32 (no grouped-query attention)

Precision: 2 bytes (FP16)

STEP 2

Per-token storage

Each layer stores a key and a value vector per KV head (the 2 below).

= 2 × Layers × KV heads × Head dim × Bytes

2 × 40 × 32 × 128 × 2 bytes

≈ 0.66 MB per token (all layers)

STEP 3

Scale across context

Total KV = per-token × Context length × Concurrent requests

0.66 MB × 2048 × 1 ≈ **1.3 GB**

With grouped-query attention (8 KV heads instead of 32) this drops 4× to ≈ 0.34 GB. Most current models use it; see the Llama 3 slides.

Total memory estimate

Combining all components gives us the complete memory footprint for our example 10B parameter model.

20
MODEL WEIGHTS (GB)
FP16 precision storage

+

1.3
KV-CACHE (GB)
2048-token context, one request

+

4.3
OVERHEAD (GB)
~20%: activations, CUDA context, buffers

=

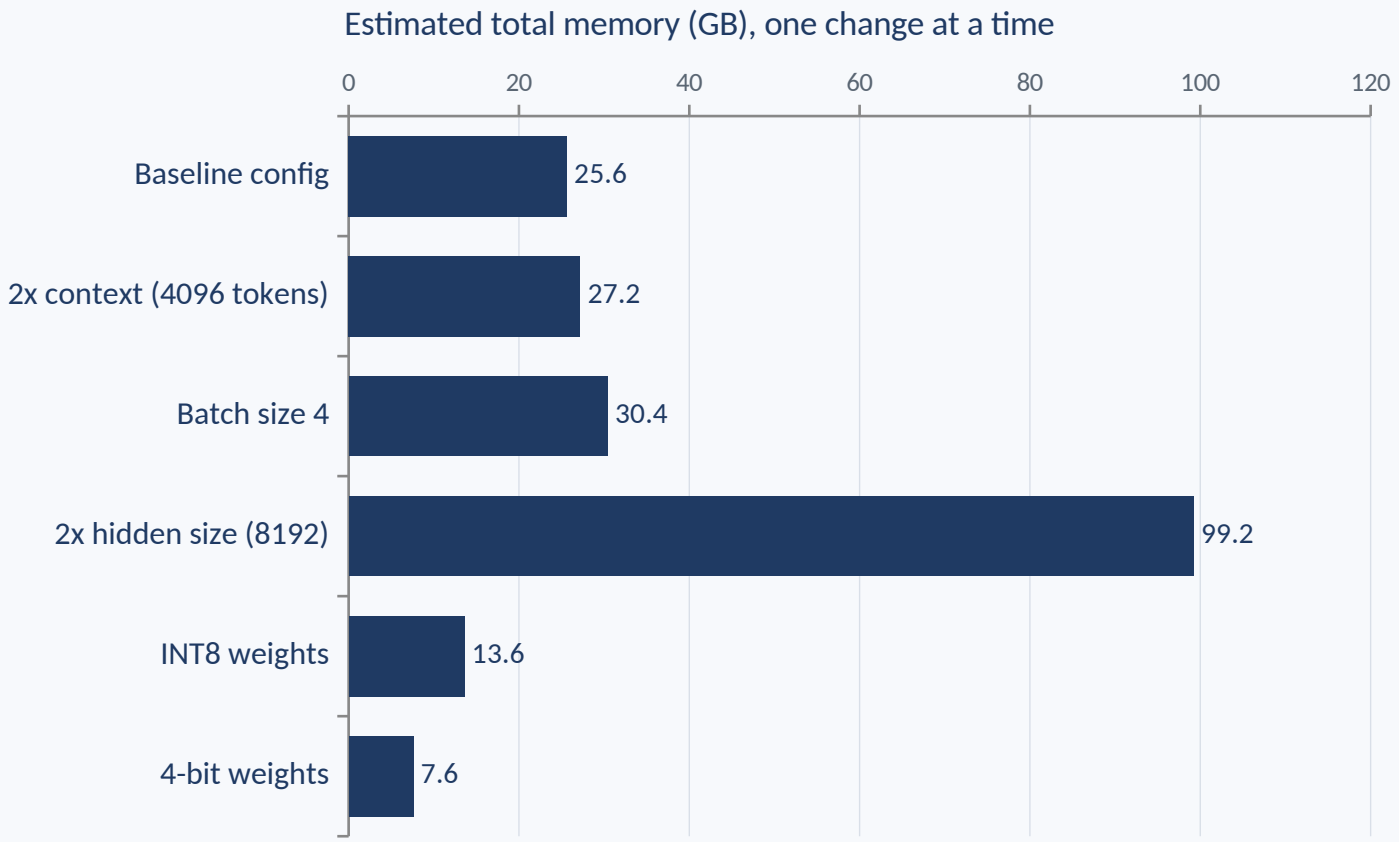
25.6
TOTAL ESTIMATE (GB)
Baseline memory requirement

GPU Selection: You'll need **comfortably more than 26 GB**. A 32 GB card would run it, with little room for longer contexts or more users.

Recommended: 40GB A100 or 48GB A6000 for this workload with comfortable headroom.

SENSITIVITY

Impact of varying parameters



Small changes to key parameters can dramatically affect memory requirements.

KEY INSIGHTS

Doubling hidden size roughly quadruples the parameters, so weights jump to ~80 GB.

Context length and batch size scale only the KV-cache, linearly.

INT8 cuts the total ~47%; 4-bit ~70%.

Where the numbers come from: config.json

CONFIG.JSON (EXCERPT)

```
{
  "hidden_size": 4096,
  "num_hidden_layers": 32,
  "num_attention_heads": 32,
  "num_key_value_heads": 8,
  "intermediate_size": 14336,
  "vocab_size": 128256,
  "torch_dtype": "bfloat16"
}
```

Find it in the model's Files tab on Hugging Face.

num_hidden_layers	Layers: each one keeps its own slice of the KV-cache
num_key_value_heads	KV heads: fewer than the 32 attention heads means grouped-query attention
hidden ÷ attention heads	Head dimension: 4096 ÷ 32 = 128 values per key or value vector
torch_dtype	bfloat16 = 2 bytes per parameter and per cached value
Model card	Parameter count: 8.03 billion

A REAL MODEL · LLAMA 3 8B

Grouped-query attention shrinks the KV-cache

MULTI-HEAD ATTENTION · 32 KV HEADS



GROUPED-QUERY ATTENTION · 8 KV HEADS



Query heads (not cached) Key/value heads (cached)

Each group of 4 query heads shares one key/value head, so there is 4× less to cache, with little loss in quality.

LLAMA 3 8B, PER TOKEN

If it used MHA

0.52 MB

`2 × 32 × 32 × 128 × 2 bytes`

With GQA (actual)

0.13 MB

`2 × 32 × 8 × 128 × 2 bytes`

One 8K-token conversation

1.1 GB

`down from 4.3 GB: 4× smaller`

A REAL MODEL · LLAMA 3 8B

Serving Llama 3 8B on a 24 GB GPU

Same method as the worked example, now with numbers from the real config.

STEP 1

Weights

8.03 billion parameters × 2 bytes (BF16)

≈ 16.1 GB

STEP 2

KV-cache per token

2 × 32 layers × 8 KV heads × 128 × 2 bytes

≈ 0.13 MB per token

One 8K-token conversation ≈ **1.1 GB**

STEP 3

What fits in 24 GB?

Usable after 20% overhead: 24 ÷ 1.2 = 20 GB

Left for KV-cache: 20 − 16.1 ≈ 3.9 GB

3.9 GB ÷ 0.13 MB ≈ **30,000 tokens**

That's about 3 conversations at the full 8K context, or about 15 at 2K. Context length is a budget you spend.

YOUR TURN

Estimate memory for Llama 3 70B

GIVEN (FROM CONFIG.JSON)

parameters:	70.6B
layers:	80
attn heads:	64
KV heads:	8
hidden size:	8192
dtype:	bfloat16
context:	8,192 tokens

1. **How much memory do the BF16 weights need?**
2. **How large is the KV-cache per token, and for one 8K conversation?**
3. **With 20% overhead, does it fit on one 80 GB H100?**
4. **If not, name two ways to make it fit.**

Llama 3 70B needs more than one GPU

1

141

WEIGHTS (GB)

70.6B × 2 bytes

2

0.33

KV PER TOKEN (MB)

2 × 80 × 8 × 128 × 2 bytes; ≈ 2.7 GB per 8K conversation

3

173

TOTAL (GB)

(141 + 2.7) × 1.2. Does not fit on one 80 GB GPU

4

2 fixes

MAKE IT FIT

Split across 4 GPUs with tensor parallelism, or 4-bit weights: (35 + 2.7) × 1.2 ≈ 46 GB on one GPU

Why 4 GPUs and not 3? Tensor parallelism splits attention heads evenly, so the GPU count must divide the head count (64 query, 8 KV heads). 2, 4, or 8 work; 3 does not.

CHECK YOUR ESTIMATE

Measure it on a real GPU

PYTHON · VLLM

```
from vllm import LLM

llm = LLM(
  model="meta-llama/
    Meta-Llama-3-8B-Instruct",
  max_model_len=8192,
  gpu_memory_utilization=0.90,
)

$ nvidia-smi    # ~90% in use
```

1

Load the model

vLLM loads ~16 GB of weights, then claims GPU memory up to the 90% limit.

2

Read the startup log

It reports how much memory went to the KV-cache and the maximum concurrency at your context length.

3

Compare to your estimate

Differences come from CUDA graphs, activation buffers, and the 10% it leaves free.

Gotcha: nvidia-smi shows memory reserved, not memory needed. vLLM pre-allocates and fills the rest with KV-cache blocks.

BEYOND INFERENCE

Training needs about 8× more memory per parameter

Inference
2 bytes / param



Full training
16 bytes / param



Plus activations, which grow with batch size × sequence length.

16 GB

LLAMA 3 8B INFERENCE

BF16 weights only, before KV-cache

~128 GB

LLAMA 3 8B FULL FINE-TUNE

8B × 16 bytes, before activations

LoRA

WHY ADAPTERS EXIST

Train small adapters on a frozen base. QLoRA's 4-bit base fits an 8B fine-tune on one 24 GB GPU.

SUMMARY

Plan memory before you deploy

Proper memory planning is essential for successful LLM inference deployment. Use these calculations as a foundation for your infrastructure decisions.

Memory Components Dominate

Model weights and KV-cache are the primary consumers. KV-cache may exceed weight memory for long contexts or many users.

Key Optimization Levers

Precision, grouped-query attention, context length, and batch size set the total; parallelism spreads it across GPUs.

Calculate Before Deploying

Read config.json and apply the formulas here before you buy or rent hardware. Avoid costly OOM errors in production.

Plan for Growth

Always include buffer margin (10-20%) for overhead, longer contexts, larger models, and future scaling needs.

Next steps: Apply these formulas to your specific model architecture. Verify with vLLM's startup log or a profiler. Iterate on quantization and parallelism strategies to find the optimal memory-performance balance.